



Proposal Teknis Proyek Akhir

Penerapan Keamanan Informasi pada Platform Manajemen Lowongan Magang Berbasis Web (CareerFlow)

Kelompok 5 :

Hanifah Syahidah	G6401231067	K2
Khalisa Rana Putri	G6401231141	K2
Aliya Shahira	G6401231116	K2

I. Deskripsi Sistem

A. Latar Belakang

CareerFlow adalah platform berbasis web yang dirancang untuk memfasilitasi proses pencarian dan pengelolaan lowongan magang serta Management Trainee (MT) bagi mahasiswa. Sistem ini menghubungkan mahasiswa yang sedang mencari pengalaman kerja dengan informasi lowongan yang dikelola oleh administrator platform. Pada konteks keamanan informasi, CareerFlow menangani data sensitif pengguna seperti identitas mahasiswa, data autentikasi, serta riwayat lamaran pekerjaan. Oleh karena itu, implementasi protokol keamanan yang memadai menjadi kebutuhan utama dalam pengembangan sistem ini.

B. Tujuan Sistem

1. Menyediakan platform terpusat untuk pencarian lowongan magang dan MT bagi mahasiswa.
2. Memberikan antarmuka admin untuk mengelola data lowongan secara aman.
3. Mengimplementasikan sistem autentikasi dan otorisasi berbasis peran (Role-Based Access Control).
4. Melindungi data pengguna dengan mekanisme enkripsi dan hashing yang standar.
5. Menyediakan fitur tracking lamaran untuk mahasiswa.

C. Pengguna Sistem

Peran	Deskripsi	Hak Akses
Admin	Pengelola platform yang bertugas menambah, mengedit, dan menghapus data lowongan magang.	CRUD Lowongan, manajemen data platform
Mahasiswa	Pengguna umum yang mencari dan melamar lowongan magang atau MT.	Browse lowongan, melamar, tracking status lamaran, kelola profil

II. Modul Sistem

A. Modul Autentikasi

Modul ini menangani seluruh proses verifikasi identitas pengguna sebelum mengakses sistem.

B. Registrasi Mahasiswa

1. Input: nama, email, NIM, program studi, password.
2. Validasi field wajib menggunakan validator.py.
3. Pengecekan duplikasi email dan NIM sebelum penyimpanan.
4. Password di-hash menggunakan Werkzeug generate_password_hash (PBKDF2-SHA256) sebelum disimpan ke database.
5. Proses verifikasi email melalui OTP 6 digit yang dikirim ke email pendaftar.

C. Login

1. Mendukung login untuk dua peran: Admin dan Mahasiswa.
2. Verifikasi password menggunakan check_password_hash.
3. Jika autentikasi berhasil, sistem menerbitkan JWT (JSON Web Token) yang mengandung identity dan role pengguna.
4. Token digunakan untuk mengamankan endpoint yang memerlukan autentikasi.

D. Lupa Password

1. Pengguna memasukkan email terdaftar.
2. Sistem memvalidasi keberadaan email di database.
3. OTP 6 digit dikirimkan ke email dan disimpan sementara di tabel otp_code.
4. OTP memiliki masa berlaku 10 menit dan hanya dapat digunakan satu kali (is_used flag).
5. Setelah OTP terverifikasi, pengguna dapat mengatur password baru yang langsung di-hash.

E. Modul Manajemen Lowongan

1. Modul ini dikelola eksklusif oleh Admin dan menangani siklus hidup data lowongan magang.
2. Tambah Lowongan: Admin mengisi form lengkap termasuk judul, perusahaan, tanggal, lokasi, gaji, tipe pekerjaan, deskripsi, dan tanggung jawab.
3. Edit Lowongan: Perubahan data lowongan yang sudah ada melalui endpoint PUT /admin/lowongan/<id>.
4. Hapus Lowongan: Penghapusan permanen data lowongan melalui endpoint DELETE /admin/lowongan/<id>.
5. Seluruh operasi CRUD lowongan dilindungi JWT dan pengecekan role admin pada level controller.

F. Modul Pencarian & Browse Lowongan

1. Modul publik yang dapat diakses mahasiswa untuk menemukan lowongan yang relevan.
2. Pencarian berdasarkan keyword (judul atau nama perusahaan) melalui query parameter.
3. Filter berdasarkan tipe lowongan (internship atau MT).
4. Tampilan detail lengkap per lowongan termasuk deskripsi peran dan tanggung jawab.

G. Modul Lamaran & Tracking

1. Modul yang memungkinkan mahasiswa untuk melamar dan memantau status lamarannya.
2. Mahasiswa dapat mengirim lamaran beserta dokumen CV.
3. Setiap lamaran memiliki status yang dapat diperbarui (Menunggu, Diproses, Diterima, Ditolak).
4. Dashboard tracking menampilkan seluruh lamaran yang pernah dikirim beserta statusnya.

H. Modul Profil

1. Modul pengelolaan data diri mahasiswa yang sudah terdaftar di sistem.
2. Mahasiswa dapat melihat dan memperbarui informasi profil.
3. Perubahan email memerlukan verifikasi password untuk keamanan tambahan.
4. Bisa autentikasi untuk perubahan email juga forgot password dari profile

III. Arsitektur Sistem

A. Stack Teknologi

Layer	Teknologi	Keterangan
Frontend	React.js + Vite + SCSS	Antarmuka pengguna berbasis komponen
Backend	Flask (Python)	REST API, business logic, keamanan
Database	SQLite / PostgreSQL	Penyimpanan data persisten
Autentikasi	JWT (Flask-JWT-Extended)	Token-based authentication
Password Hashing	Werkzeug (PBKDF2-SHA256)	Hashing & salting password
OTP	Random 6-digit + DB storage	Verifikasi email & reset password
CORS	Flask-CORS	Cross-origin request control

B. Arsitektur Berlapis

1. Sistem CareerFlow menggunakan arsitektur berlapis (layered architecture) yang memisahkan tanggung jawab setiap komponen:
2. Presentation Layer: React.js menangani tampilan dan interaksi pengguna.
3. API Layer: Blueprint Flask (auth_routes, admin_routes, magang_routes) mendefinisikan endpoint.
4. Controller Layer: AuthController dan MagangController menangani request/response dan validasi role.
5. Service Layer: AuthService dan MagangService mengandung logika bisnis utama.

6. Data Layer: SQLAlchemy ORM mengelola interaksi dengan database.
7. **Manajemen Kunci & Token**
8. JWT Secret Key disimpan sebagai environment variable (tidak di-hardcode dalam kode).
9. File .env dikecualikan dari version control melalui .gitignore.
10. Token JWT mengandung identity (user ID) dan additional claims (role) yang divalidasi setiap request.
11. Token tidak disimpan di server (stateless), validasi dilakukan via signature verification.

IV. Implementasi Keamanan

A. Password Hashing & Salting (Sudah Diimplementasi)

Password pengguna tidak pernah disimpan dalam bentuk plaintext. Sistem menggunakan Werkzeug Security yang mengimplementasikan PBKDF2-SHA256 dengan salt acak:

1. Saat registrasi: `generate_password_hash(data['password'])` menghasilkan hash unik setiap kali meski password sama.
2. Saat login: `check_password_hash(user.password, password)` memverifikasi tanpa perlu decode hash.
3. Salt di-embed otomatis dalam hash string, sehingga setiap hash bersifat unik.

B. OTP (One-Time Password) (Sudah Diimplementasi)

1. OTP 6 digit dibangkitkan menggunakan `random.choices(string.digits, k=6)`.
2. Setiap OTP terikat pada email, purpose (signup/forgot), dan timestamp pembuatan.
3. OTP kadaluarsa setelah 10 menit dan ditandai `is_used=True` setelah digunakan.
4. OTP lama yang belum kadaluarsa dibatalkan saat request OTP baru dikirim.

C. JWT Authentication (Sudah Diimplementasi)

1. Seluruh endpoint sensitif dilindungi dekorator `@jwt_required()`.
2. Payload JWT mengandung role pengguna untuk keperluan otorisasi.
3. Controller memvalidasi role sebelum memproses request (admin-only endpoints).

4. Input Validation (Sudah Diimplementasi)

5. Fungsi `validate_required_fields()` memastikan semua field wajib terisi sebelum diproses.
6. Validasi dilakukan di level controller sebelum meneruskan data ke service layer.

D. Enkripsi Data at Rest (Enkripsi Simetris) (Sudah Diimplementasi)

1. Sesuai dengan protokol keamanan sistem, data log sensitif (seperti alamat IP pengguna) dilindungi di dalam database menggunakan algoritma AES via library `Python cryptography.fernet`.
2. Fungsi `encrypt_data()` diterapkan pada *service layer* sebelum rekaman masuk ke tabel `ActivityLog`. Algoritma ini menjamin kerahasiaan (*Confidentiality*) dengan menambahkan *timestamp* dan *Initialization Vector* (IV) acak pada setiap eksekusi sehingga hasil *ciphertext* selalu unik dan mencegah *hacker* melakukan *pattern analysis*.

3. Fungsi `decrypt_data()` digunakan saat sistem perlu menyajikan data kepada pihak yang berwenang, memastikan data IP kembali menjadi *plaintext* yang dapat dibaca.
4. Kunci rahasia kriptografi (`AES_SECRET_KEY`) tidak pernah di-*hardcode* ke dalam *source code*. Kunci tersebut dikelola melalui *Environment Variables* (`.env`) dan dimasukkan ke dalam `.gitignore` agar tidak ikut ter-*commit* ke repositori Git.

E. Digital Signature (Direncanakan - Minggu 9-10)

1. Untuk memenuhi kebutuhan non-repudiation, implementasi digital signature direncanakan pada fase berikutnya:
2. Menggunakan library Python cryptography dengan algoritma RSA atau ECDSA.
3. Setiap operasi CRUD lowongan oleh admin akan di-sign menggunakan private key admin.
4. Signature disimpan di database bersama data transaksi.
5. Endpoint verifikasi akan disediakan untuk membuktikan keaslian dan integritas data.
6. Key pair (public/private) di-generate saat setup dan private key disimpan secara aman di server.

V. Model Data

Sistem CareerFlow menggunakan lima entitas utama dalam database:

Entitas	Primary Key	Atribut Utama
Admin	idUser (UUID)	nama, email, password (hashed), idAdmin
Mahasiswa	idUser (UUID)	nama, email, password (hashed), nim, programStudi
LowonganMagang	id (UUID)	title, company, dateRange, type, location, salary, about, responsibilities, kuota, idAdmin
Lamaran	idLamaran (UUID)	tanggalApply, statusLamaran, dokumenCV, idLowongan, nimMahasiswa
OtpCode	id (Integer)	email, otp, purpose, created_at, is_used

VI. API Endpoint

Authentication Endpoints

Method	Endpoint	Auth	Deskripsi
POST	/signup/send-otp	Public	Kirim OTP untuk registrasi
POST	/signup/verify-otp	Public	Verifikasi OTP registrasi
POST	/signup	Public	Registrasi akun mahasiswa
POST	/login	Public	Login, mendapatkan JWT
POST	/forgot-password/send-otp	Public	Kirim OTP reset password

POST	/forgot-password/verify-otp	Public	Verifikasi OTP reset password
POST	/forgot-password/reset	Public	Reset password baru
GET	/profile	JWT Required	Ambil data profil pengguna

Admin Endpoints

Method	Endpoint	Auth	Deskripsi
POST	/admin/lowongan	JWT + Role Admin	Tambah lowongan baru
PUT	/admin/lowongan/<id>	JWT + Role Admin	Update data lowongan
DELETE	/admin/lowongan/<id>	JWT + Role Admin	Hapus lowongan

Mahasiswa & Public Endpoints

Method	Endpoint	Auth	Deskripsi
GET	/lowongan	Public	Daftar semua lowongan (dengan filter)
GET	/lowongan/<id>	Public	Detail lowongan
POST	/lamaran	JWT + Mahasiswa	Kirim lamaran
GET	/mahasiswa/dashboard	JWT + Mahasiswa	Dashboard tracking lamaran
PUT	/lamaran/<id>/status	JWT + Mahasiswa	Update status lamaran